



IW AI Core Platform — Architecture Map

Type: Architecture **Version:** v1 **Status:** Planned **Generated:** 2026-06-15 13:15:33

IW AI Core Platform — Architecture Map

Editorial category: technical

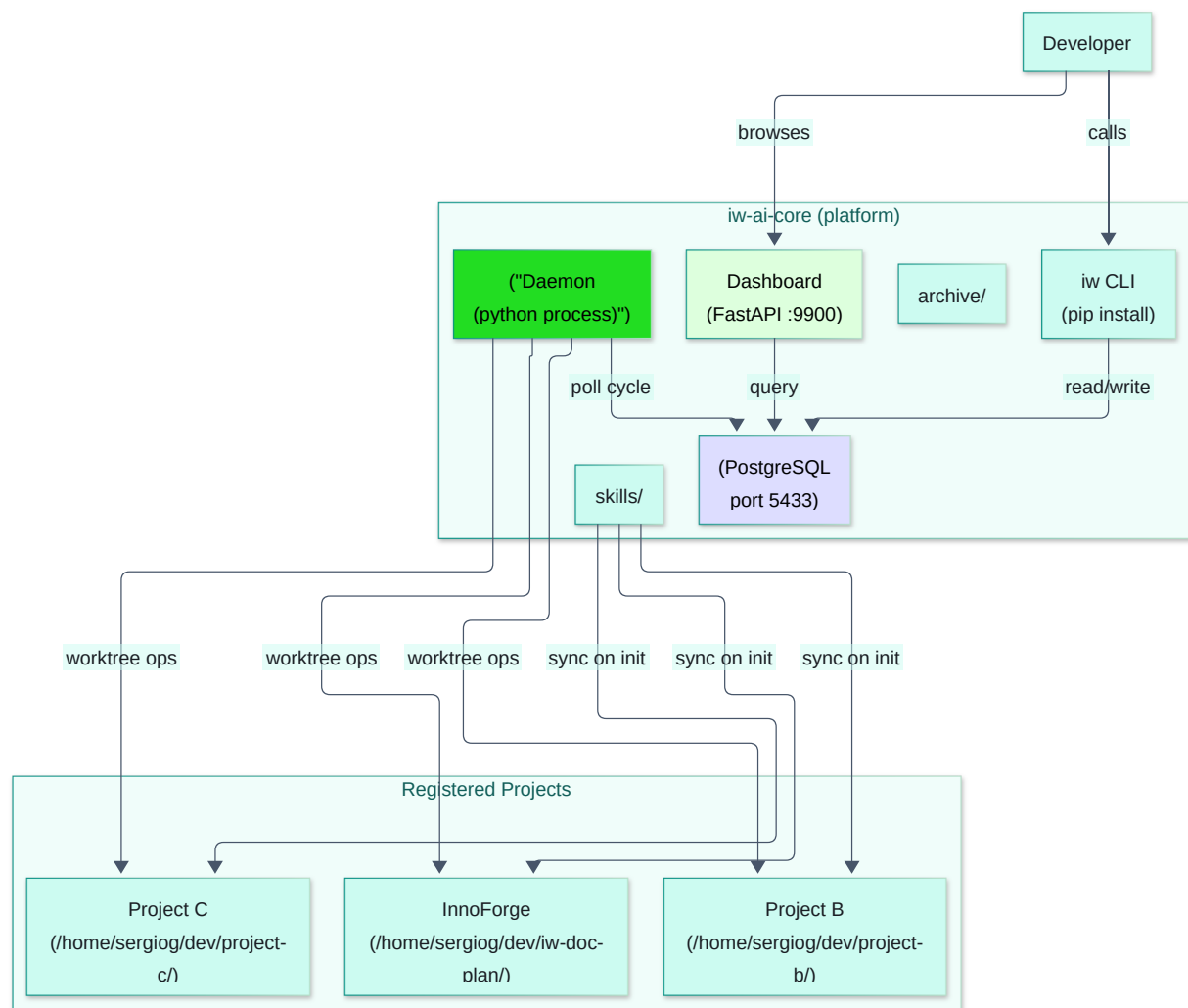
Generated: 2026-05-11

Executive Summary

IW AI Core is a standalone orchestration platform that drives AI-assisted development across multiple software projects. It replaces file-based tracking (markdown ID files, JSON manifests) with a PostgreSQL-backed system where the database is the single source of truth for all operational state: IDs, step statuses, batch execution, agent heartbeats, and merge queues. A single daemon process polls the database every 60 seconds, creates git worktrees for agents, and manages the full lifecycle from work item creation through merge and archive. The FastAPI dashboard on port 9900 provides the human interface across all registered projects.

System Layout

Why this diagram? Use this to understand which physical components exist, how the iw-ai-core platform relates to managed projects, and where each type of traffic (human UI, agent CLI, daemon orchestration) flows.



Core Components

The Database (PostgreSQL on port 5433)

Every table uses `project_id` for multi-project isolation. Composite primary keys `(project_id, id)` ensure no cross-project collisions. Atomic ID allocation uses `FOR UPDATE` row locks — no race conditions under concurrent agent access.

Core tables:

Table	Purpose
<code>projects</code>	Registration: <code>id</code> , <code>display_name</code> , <code>repo_root</code> , <code>enabled</code>
<code>id_sequences</code>	Atomic ID allocation: <code>(project_id, prefix) → next_number</code>
<code>work_items</code>	Design doc, title, status (<code>draft</code> → <code>approved</code> → <code>in_progress</code> → <code>completed</code> <code>failed</code>), phase, Tier-1 content
<code>workflow_steps</code>	Step number, agent, status, report content (Tier 1)
<code>step_runs</code>	Execution control plane: PID, <code>pid_alive</code> , heartbeat, timeout, command (for restart), <code>exit_code</code>
<code>batches</code>	Batch-level status, <code>max_parallel</code> , <code>auto_publish</code>

Table	Purpose
<code>batch_items</code>	Per-item status in a batch: <code>pending→setting_up→executing→completed→merged failed</code>
<code>daemon_events</code>	Audit trail: <code>event_type</code> , <code>entity_id</code> , <code>message</code> , <code>metadata</code>

Tier 1 storage: design docs and reports stored as `TEXT` in `work_items` and `workflow_steps`. Always viewable, full-text searchable via `tsvector`.

Tier 2 storage: full artifacts (prompts, logs, evidences) compressed to `archive/<project>/<item_id>.tar.zst`. Not in git. On-demand extraction to `/tmp`.

The Daemon

Single Python process. Single-threaded poll loop every 60 seconds. Manages all registered projects from a single process.

Responsibilities:

- Poll for `approved` / `executing` batches across all projects
- Create git worktrees via `executor/worktree_setup.sh`
- Launch LLM agents (opencode or claude-code) for each step
- Monitor running steps: `kill -0` for aliveness, heartbeat tracking, stall detection
- Process merge queue one project at a time (squash-merge via `executor/worktree_commit.sh`)
- Track git status per project
- Emit events to `daemon_events` table
- Poll `DocGenerationJob` and `CodeIndexJob` tables

Stall detection: if a running step's `last_heartbeat` is more than 600 seconds old, the daemon marks it `stalled`.

Timeout: per-step-type configurable timeouts (default: 45 min for implementation, 30 min for code review, 10 min for lint).

The Dashboard (FastAPI on port 9900)

No React SPA. No build pipeline. Jinja2 templates + htmx for partial page updates + SSE for live notifications.

Key pages:

- `/project/<project>/queue` — pending work items, design docs, approve button
- `/project/<project>/batches` — all batches with status
- `/project/<project>/batch/<batch_id>` — timeline, Gantt-style execution view
- `/project/<project>/item/<item_id>` — step pipeline, one-click restart/kill/skip
- `/system/running` — all active steps across all projects with live duration counters
- `/system/all-active` — cross-project view of in-flight items

One-click actions (all DB-backed, daemon picks up within 60s):

- **Kill:** SIGTERM to the stored PID, mark `step_runs.status='killed'`
- **Restart:** re-insert `step_runs` row with same `command` + `worktree_path`, daemon re-launches
- **Skip:** mark step `completed`, daemon advances

The `iw` CLI

Bridge between LLM agents and the database. Installed via `pip install -e /path/to/iw-ai-core`. Available on PATH from any directory. Finds the current project by looking for `.iw-orch.json` up the directory tree.

Command groups:

Group	Commands	Used by
ID management	<code>iw next-id</code> , <code>iw current-project</code>	Skills (LLM agents)
Work items	<code>iw register</code> , <code>iw approve</code> , <code>iw archive</code>	Skills, human
Steps	<code>iw step-start</code> , <code>iw step-done</code> , <code>iw step-fail</code>	Agents (via <code>/execute</code> skill)
Batches	<code>iw batch-create</code> , <code>iw batch-approve</code> , <code>iw batch-status</code>	Skills, human
Migration lock	<code>iw migration-lock acquire/release/status</code>	Agents
Skills	<code>iw sync-skills</code> , <code>iw init-project</code>	Human
System	<code>iw daemon start/stop/status</code> , <code>iw projects list</code>	Human
Doc jobs	<code>iw doc-job-status</code> , <code>iw doc-job-done</code> , <code>iw doc-update</code>	Platform (DocJobPoller)

Executor Scripts

Deterministic bash. No LLM involvement. Project-agnostic.

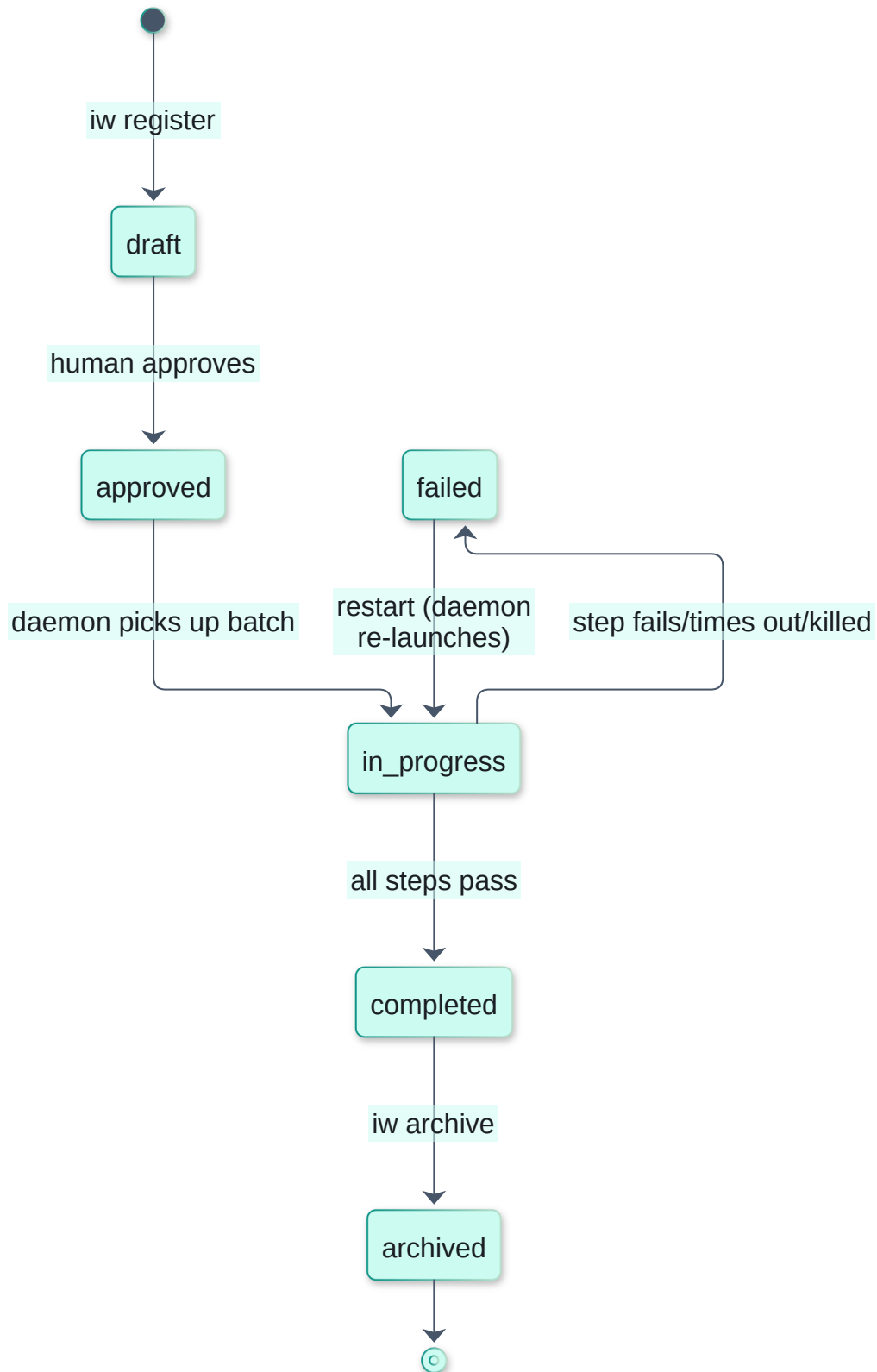
Script	Purpose
<code>worktree_setup.sh</code>	<code>git worktree add</code> , install deps, sync skills, update DB to <code>setting_up</code>
<code>worktree_commit.sh</code>	Squash-merge worktree branch to main, update <code>batch_items.merged</code> , clean up worktree
<code>step_executor.sh</code>	Launch LLM agent for a step, handle timeouts, parse results (legacy — daemon now handles launch directly)

Skills Distribution

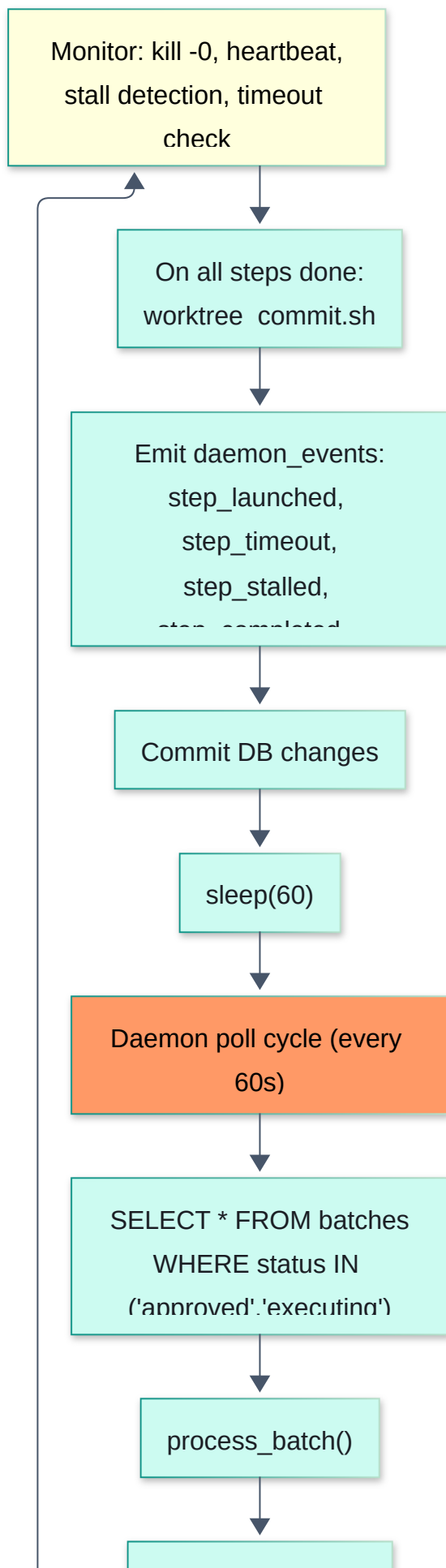
Master copies live in `iw-ai-core/skills/`. `iw sync-skills` copies them to each project's `.claude/skills/`. Projects can override any skill by creating their own version — the sync skips overrides and records them in `.iw-skills-lock.json`.

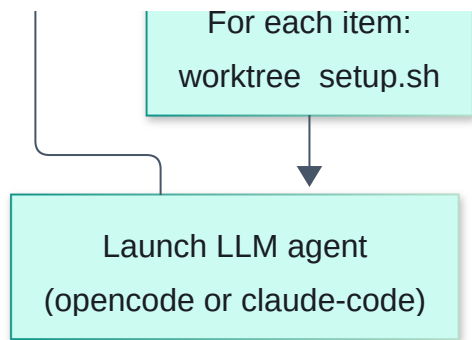
Data Flow

Work Item Lifecycle

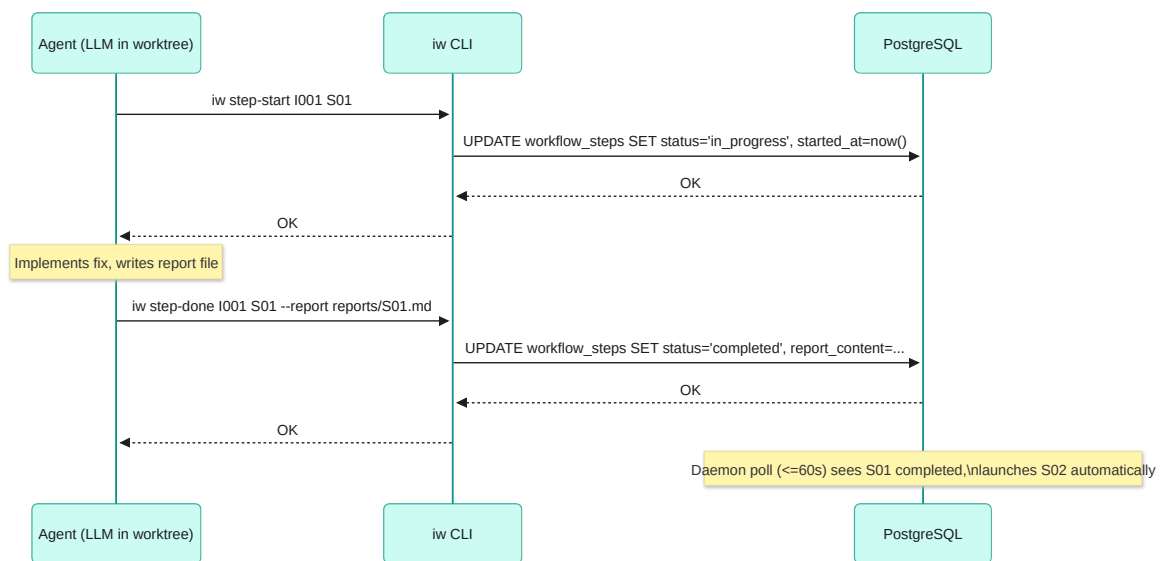


Step Execution (Daemon Poll Loop)

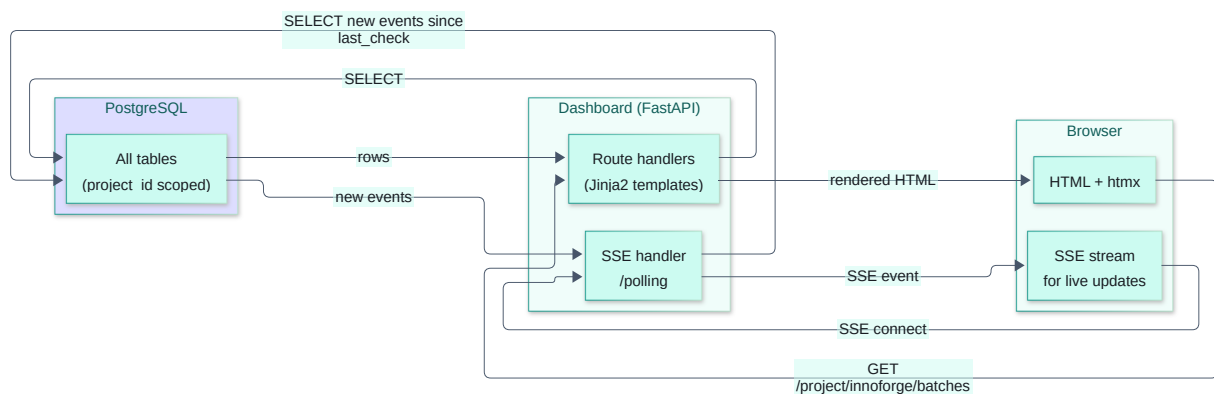




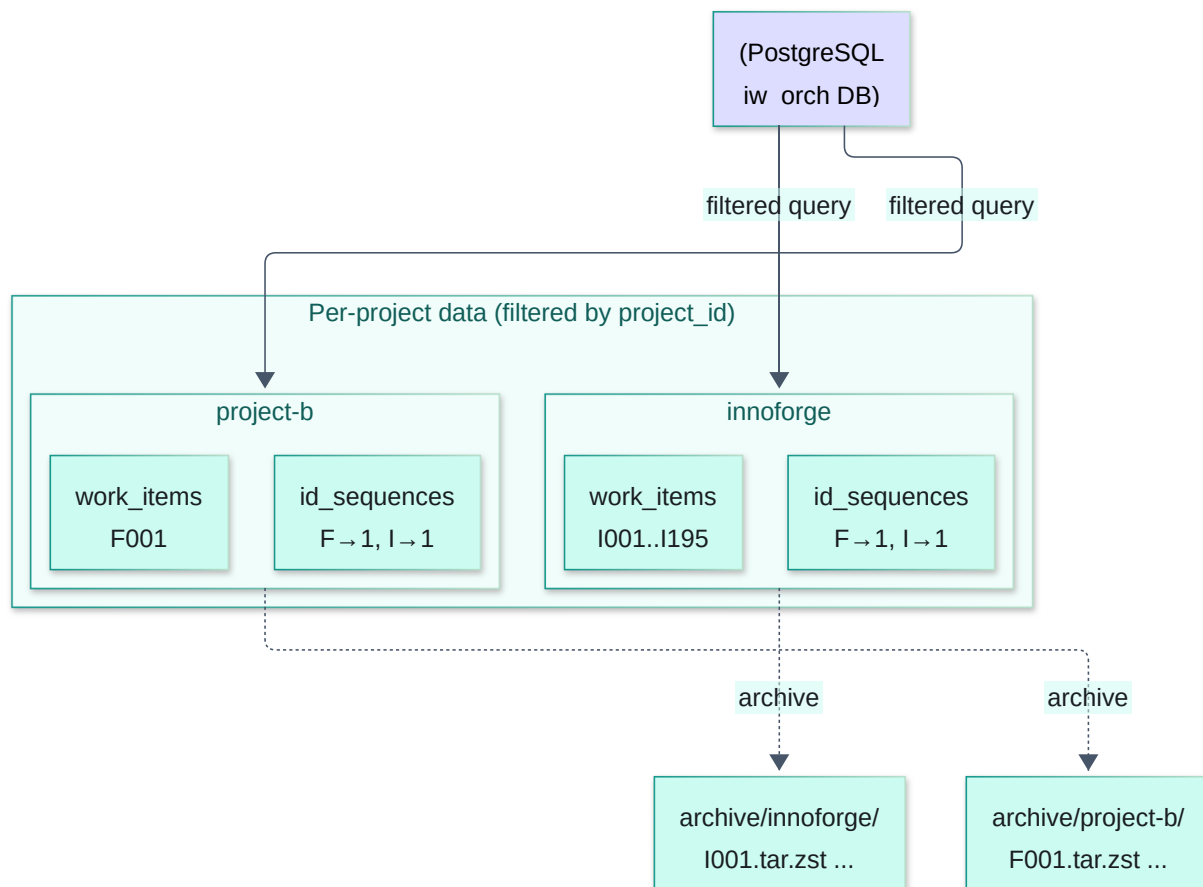
Agent-to-Database Communication



Dashboard Data Flow



Multi-Project Isolation



Isolation guarantees:

- Every table query includes `project_id` filter — no cross-project data leaks
- ID sequences are per-project: InnoForge `I001` never collides with project-b's `I001`
- Worktrees are under each project's own `.worktrees/` directory — filesystem-level isolation
- Merge queues are per-project: InnoForge merges never block project-b
- LLM quota is tracked globally (same account) but execution is per-project

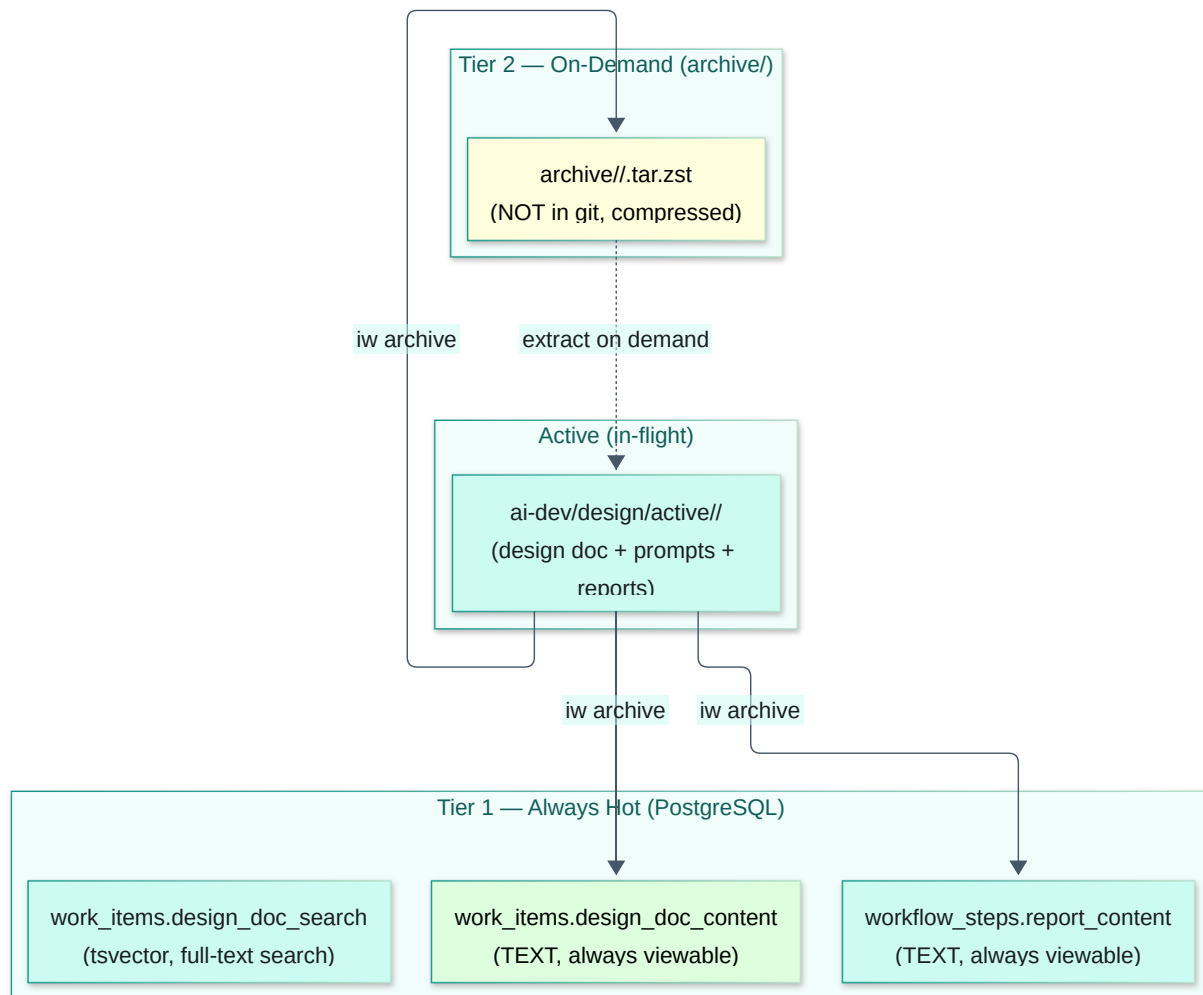
Project Registration

Each project self-describes with `.iw-orch.json` in its repo root:

```
{
  "project_id": "innoforge",
  "display_name": "InnoForge Document Platform",
  "repo_root": "/home/sergiog/dev/iw-doc-plan/main/iw-doc-plan",
  "id_prefixes": { "Feature": "F", "Issue": "I", "ChangeRequest": "CR" },
  "worktree_base": ".worktrees",
  "ai_dev_dir": "ai-dev",
  "cli_tool": "opencode",
  "quality_gates": ["ruff", "mypy", "pytest", "coverage", "semgrep"],
  "max_parallel": 4,
  "branch_prefix": "agent"
}
```

The central `projects.toml` in `iw-ai-core` links projects to the daemon. Daemon reloads on SIGHUP (`./ai-core.sh daemon reload`).

Two-Tier Storage Model



Tier 1 (always viewable in dashboard, full-text searchable):

- `work_items.design_doc_content` — full markdown of design document
- `workflow_steps.report_content` — full report markdown per step
- `work_items.summary` — AI-generated 2-3 line summary for list views

Tier 2 (on-demand extraction, not in git):

- Prompts, evidences, screenshots, raw logs
- Compressed as `.tar.zst` in `iw-ai-core/archive/<project>/`
- Extracted to `/tmp/iw-archive-view/<project>/<id>/` on request, TTL-cleaned after 10 min inactivity

Active work items (in-flight, being executed) stay in the project repo at `ai-dev/design/active/<id>/`. Agents need filesystem access during execution.

Step Runs — Execution Control Plane

Every launch (or re-launch) of a step creates a new row in `step_runs` :

Field	Purpose	Who writes
<code>pid</code>	OS process ID — enables kill, alive-check	Daemon (on launch)
<code>pid_alive</code>	Is process still running right now?	Daemon (every poll cycle via <code>kill -0</code>)
<code>command</code>	Exact launch command — enables one-click restart	Daemon (on launch)
<code>worktree_path</code>	Where the agent runs	Daemon (from <code>worktree_setup</code>)
<code>last_heartbeat</code>	Last confirmed activity	Daemon (every poll cycle)
<code>timeout_secs</code>	Dynamic per step type, not global constant	Daemon (set from step config)
<code>error_message</code>	Human-readable failure reason	Daemon or agent (via <code>iw step-fail --reason</code>)

Restart flow: User clicks [Restart] → dashboard inserts new `step_runs` row with same `command` + `worktree_path` + incremented `run_number` → daemon picks up `pending` status on next poll → re-executes with identical parameters.

Dynamic timeouts per step type:

Step type	Default timeout
Implementation (Backend, API, DB)	45 min
Code Review (per-agent)	30 min
Code Review (final)	40 min
Quality Validation (lint/format)	10 min
Quality Validation (tests)	20 min
Quality Validation (browser)	15 min
QV Fix (LLM-driven)	30 min

Quick Reference

Topic	Location
DB schema + DDL	<code>docs/IW_AI_Core_Database_Schema.md</code>
Daemon design + poll loop	<code>docs/IW_AI_Core_Daemon_Design.md</code>
Dashboard pages + htmx patterns	<code>docs/IW_AI_Core_Dashboard_Design.md</code>
CLI spec (every command)	<code>docs/IW_AI_Core_CLI_Spec.md</code>
Testing strategy + rules	<code>docs/IW_AI_Core_Testing_Strategy.md</code>
DB setup (production vs bootstrap)	<code>docs/IW_AI_Core_DB_Setup.md</code>

Architecture generated by `iw-doc-generator skill` — job `DOC-00059 (422950d3-d1df-49dc-ad62-4a25b1e6ef6d)`

Innovation Ways · Generated by IW AI Core

IW AI Core Platform